

CSS

Flexbox

¿Qué es Flexbox?

Es un método de diseño de página unidimensional para compaginar elementos en filas o columnas. Los elementos de contenido se ensanchan para rellenar el espacio adicional y se encogen para caber en espacios más pequeños. En este artículo expondremos todas sus características básicas. Sirve para:

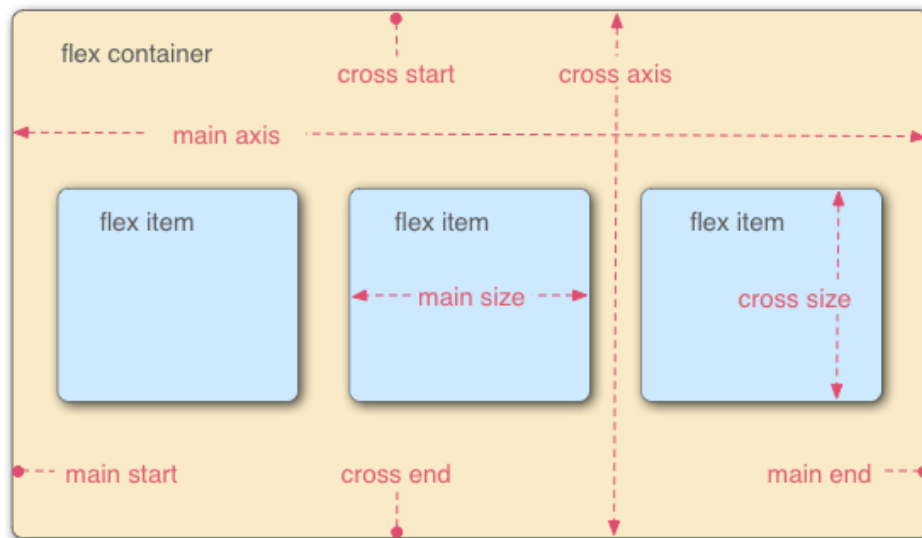
- Poner elementos **uno al lado del otro**.
- **Centrar** elementos fácilmente, tanto horizontal como verticalmente.
- **Repartir el espacio** entre los elementos.
- Hacer que los elementos se **acomoden solos** según el tamaño de la pantalla.

Se usa mucho en **menús, barras de navegación, botones, tarjetas y para centrar contenido**.

Conceptos básicos

- **El modelo flexible**

Cuando los elementos se presentan como cajas flexibles, se distribuyen con respecto a dos ejes:



- El **eje principal (main axis)** es el eje que corre en la dirección en que se colocan los elementos flexibles (por ejemplo, según se disponen las filas en una página o hacia abajo según se disponen las columnas en una página). El inicio y el final de este eje se denominan **inicio principal (main start)** y **final principal (main end)**.
- El **eje transversal (cross axis)** es el eje que corre perpendicular a la dirección en la que se colocan los elementos flexibles. El inicio y el final de este eje se denominan **inicio transversal (cross start)** y **extremo cruzado (cross end)**.
- El elemento padre que tiene establecido display: flex (el elemento [<section>](#) en nuestro ejemplo) se llama **contenedor flexible**.

- Los elementos que se presentan como cajas flexibles dentro del contenedor flexible se denominan **elementos flexibles** (son los elementos [<article>](#) de nuestro ejemplo).

Contenedor y elementos

- **Contenedor (padre):** la caja a la que se le aplica display: flex.

CSS

```
.contenedor {
  display: flex; /* activa Flexbox: los hijos quedan en fila */
}
```

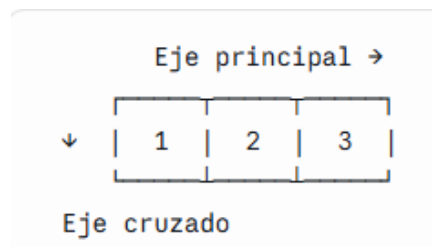
- **Elementos (hijos):** las cajas que están adentro. Se organizan automáticamente.

html

```
<div class="contenedor">      <!-- padre -->
  <div>1</div>                <!-- hijo -->
  <div>2</div>                <!-- hijo -->
  <div>3</div>                <!-- hijo -->
</div>
```

Los dos ejes

- **Eje principal:** la dirección en que se ordenan los elementos. Por defecto es horizontal (→).
- **Eje cruzado:** la dirección contraria. Por defecto es vertical (↓).



Esto es importante porque justify-content alinea en el **eje principal** y align-items alinea en el **eje cruzado**.

Propiedades del contenedor (padre)

Propiedad	Qué hace	Valores más usados
display	Activa Flexbox	flex
flex-direction	Dirección de los elementos	row, column, row-reverse, column-reverse
flex-wrap	Si los elementos bajan a otra línea cuando no caben	nowrap, wrap

Propiedad	Qué hace	Valores más usados
flex-flow	Atajo de flex-direction + flex-wrap	row wrap
justify-content	Alinea en el eje principal	flex-start, center, flex-end, space-between, space-around, space-evenly
align-items	Alinea en el eje cruzado	stretch, flex-start, center, flex-end
align-content	Alinea las líneas cuando hay varias (con wrap)	flex-start, center, space-between
gap	Espacio entre elementos	10px, 1rem

Propiedades de los elementos (hijos)

Propiedad	Qué hace	Ejemplo
order	Cambia el orden en que se muestra el elemento	order: 1;
flex-grow	Cuánto crece para llenar el espacio libre	flex-grow: 1;
flex-shrink	Cuánto se encoge si falta espacio	flex-shrink: 0;
flex-basis	Tamaño inicial del elemento	flex-basis: 200px;
flex	Atajo de grow, shrink y basis	flex: 1;
align-self	Alinea solo ese elemento en el eje cruzado	align-self: center;

Explicación de cada propiedad

flex-direction: la dirección

Valor	Resultado
row (por defecto)	En fila, de izquierda a derecha: 1 2 3
row-reverse	En fila, al revés: 3 2 1
column	En columna, de arriba hacia abajo
column-reverse	En columna, de abajo hacia arriba

justify-content: alinear en el eje principal

flex-start	1 2 3		todos al inicio
center	1 2 3		todos al centro
flex-end	1 2 3		todos al final
space-between	1 2 3		el primero y el último pegados a los bordes
space-around	1 2 3		espacio alrededor de cada uno
space-evenly	1 2 3		el mismo espacio en todas partes

align-items: alinear en el eje cruzado

Valor	Resultado
stretch (por defecto)	Los elementos se estiran para llenar el alto
flex-start	Pegados arriba
center	Centrados verticalmente
flex-end	Pegados abajo

flex-wrap: saltar de línea

- nowrap (por defecto): todos en una sola línea, aunque se aprieten.
- wrap: si no caben, bajan a la siguiente línea.

gap: separación

Pone espacio **entre** los elementos, sin tener que usar margin en cada uno.

flex: 1: repartir el espacio

Si todos los hijos tienen flex: 1, se reparten el espacio **en partes iguales**. Si uno tiene flex: 2, ocupa el **doble** que los demás.

Ejemplos:

- **1.Menú de navegación horizontal:** los enlaces quedan en fila y separados.

html

```
<nav class="menu">
  <a href="#">Inicio</a>
  <a href="#">Servicios</a>
  <a href="#">Contacto</a>
</nav>
```

CSS

```
.menu {
  display: flex;
  gap: 20px;
  background-color: #2b6cb0;
  padding: 15px;
}
.menu a {
  color: white;
  text-decoration: none;
}
```

- **2.Centrar un elemento en la pantalla:** es la forma más fácil de centrar horizontal y verticalmente a la vez.

html

```
<div class="centrar">
  <div class="caja">Estoy en el centro</div>
</div>
```

CSS

</> 

```
.centrar {
  display: flex;
  justify-content: center; /* centro horizontal */
  align-items: center; /* centro vertical */
  height: 100vh; /* alto de toda la pantalla */
}
.caja {
  background-color: #63b3ed;
  padding: 30px;
}
```

- **3.Encabezado con logo a la izquierda y menú a la derecha:** space-between empuja los elementos hacia los extremos.

html

```
<header class="encabezado">
  <h1>Clínica Dental</h1>
  <nav>
    <a href="#">Inicio</a>
    <a href="#">Citas</a>
  </nav>
</header>
```

CSS

```
.encabezado {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 20px;
  background-color: #f5f5f5;
}
```

- **4.Tarjetas que bajan de línea:** con wrap, si la pantalla es pequeña, las tarjetas pasan a la siguiente fila.

html

```
<div class="tarjetas">
  <div class="tarjeta">Limpieza</div>
  <div class="tarjeta">Ortodoncia</div>
  <div class="tarjeta">Blanqueamiento</div>
  <div class="tarjeta">Implantes</div>
</div>
```

CSS

```
.tarjetas {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
}
.tarjeta {
  width: 200px;
  padding: 20px;
  background-color: #ebf4ff;
  border: 1px solid #2b6cb0;
}
```

- **5.Elementos en columna:** flex-direction: column los pone uno debajo del otro, como un menú lateral.

html

```
<aside class="lateral">
  <a href="#">Perfil</a>
  <a href="#">Mis citas</a>
  <a href="#">Cerrar sesión</a>
</aside>
```

css

```
.lateral {
  display: flex;
  flex-direction: column;
  gap: 10px;
  width: 200px;
  padding: 15px;
  background-color: #333333;
}
.lateral a {
  color: white;
}
```

- **6.Columnas del mismo tamaño:** con flex: 1, todas las columnas se reparten el espacio por igual.

html

```
<div class="columnas">
  <div>Columna 1</div>
  <div>Columna 2</div>
  <div>Columna 3</div>
</div>
```

css

```
.columnas {
  display: flex;
  gap: 10px;
}
.columnas div {
  flex: 1;
  padding: 20px;
  background-color: #c6f6d5;
}
```

- **7.Barra de búsqueda donde el campo crece:** el campo de texto ocupa todo el espacio libre y el botón mantiene su tamaño.

html

```
<form class="buscador">
  <input type="text" placeholder="Buscar...">
  <button type="submit">Buscar</button>
</form>
```

css

```
.buscador {
  display: flex;
  gap: 5px;
}
.buscador input {
  flex-grow: 1; /* crece para llenar el espacio */
  padding: 8px;
}
```

- **8.Cambiar el orden con order:** sin cambiar el HTML, el elemento "3" se muestra de primero.

html

```
<div class="orden">
  <div>1</div>
  <div>2</div>
  <div class="primero">3</div>
</div>
```

css

```
.orden {
  display: flex;
  gap: 10px;
}
.orden div {
  padding: 20px;
  background-color: #fed7d7;
}
.primero {
  order: -1; /* un número menor lo pone antes */
}
```

- **9.Alinear un solo elemento con align-self:** todos quedan arriba menos uno, que queda abajo.

html

```
<div class="alinear">
  <div>A</div>
  <div>B</div>
  <div class="abajo">C</div>
</div>
```

css

```
.alinear {
  display: flex;
  align-items: flex-start; /* todos arriba */
  height: 150px;
  gap: 10px;
  background-color: #f5f5f5;
}
.alinear div {
  padding: 15px;
  background-color: #fefcbf;
}
.abajo {
  align-self: flex-end; /* solo este abajo */
}
```

- **10.Pie de página siempre abajo:** aunque haya poco contenido, el footer queda al final de la pantalla

html

```
<body class="pagina">
  <header>Encabezado</header>
  <main>Contenido</main>
  <footer>Pie de página</footer>
</body>
```

CSS

```
.pagina {
  display: flex;
  flex-direction: column;
  min-height: 100vh; /* mínimo el alto de la pantalla */
  margin: 0;
}
.pagina main {
  flex: 1;           /* el main crece y empuja el footer hacia abajo */
}
.pagina footer {
  background-color: #333333;
  color: white;
  padding: 15px;
}
```

Grid

¿Qué es Grid?

Una cuadrícula es un conjunto de líneas horizontales y verticales que se intersectan - un grupo define columnas y el otro filas. Los elementos se pueden colocar en la cuadrícula respetando estas columnas y filas. El diseño de cuadrícula CSS tiene las siguientes características:

- **Tamaños fijos y flexibles**

Puede crear una cuadrícula con tamaños fijos, utilizando píxeles, por ejemplo. También se puede crear una cuadrícula utilizando tamaños flexibles con porcentajes o con la nueva unidad de medida fr (fracción), diseñada para este propósito.

- **Posicionamiento de elementos**

Puede colocar elementos en una ubicación precisa en la cuadrícula utilizando números de línea, nombres o seleccionando un área de la cuadrícula. Grid también contiene un algoritmo para controlar la ubicación de elementos que no tienen una posición explícita en la cuadrícula.

- **Creación de líneas adicionales para alojar contenido**

Puede definir una cuadrícula explícita con grid layout. La especificación Grid Layout es lo suficientemente flexible como para permitir agregar filas y columnas adicionales cuando sea necesario. Características como la adición de "tantas columnas como caben en un contenedor" también fueron incluidas.

- **Control de alineación**

Grid contiene características de alineación para poder controlar la forma cómo se alinean los elementos una vez colocados en un área de cuadrícula y cómo está alineada toda la cuadrícula.

- **Control de contenido superpuesto**

Se puede colocar más de un elemento en una celda de la cuadrícula o área, las cuales pueden solaparse o superponerse total o parcialmente entre sí. Esta estratificación puede ser controlada con la propiedad z-index.

Grid es una poderosa especificación que, cuando se combina con otras partes de CSS como flexbox, puede ayudarle a crear diseños que antes eran imposibles de construir en CSS. Todo comienza creando una cuadrícula en su contenedor de cuadrícula.

Sirve para:

- Armar la **estructura completa** de una página (encabezado, barra lateral, contenido, pie de página).
- Crear **galerías** de imágenes o productos.
- Hacer que un elemento **ocupe varias filas o columnas**.
- Diseños que se **adaptan solos** al tamaño de la pantalla.

Conceptos básicos

- **Contenedor y elementos**

Igual que en Flexbox: el **contenedor (padre)** lleva display: grid y los **elementos (hijos)** se acomodan en la cuadrícula.

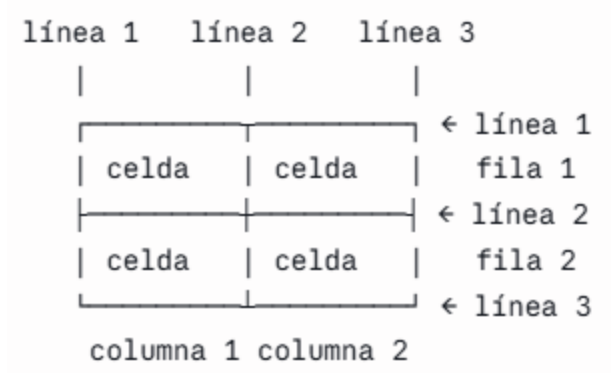
```
html

<div class="contenedor">      <!-- padre -->
  <div>1</div>                <!-- hijo -->
  <div>2</div>
  <div>3</div>
</div>
```

```
CSS

.contenedor {
  display: grid;                /* activa Grid */
  grid-template-columns: 100px 100px; /* 2 columnas de 100px */
}
```

- **Partes de la cuadrícula**



- **Filas (rows):** las líneas horizontales de celdas.
- **Columnas (columns):** las líneas verticales de celdas.
- **Celda:** el cruce entre una fila y una columna.
- **Líneas:** los bordes que separan filas y columnas. Se numeran desde 1.
- **Área:** un grupo de celdas unidas.

- **La unidad fr**

fr significa **fracción del espacio disponible**. Es la unidad más usada en Grid:

- 1fr 1fr 1fr: 3 columnas iguales.
- 2fr 1fr: la primera columna es el doble de ancha que la segunda.

Propiedades del contenedor (padre)

Propiedad	Qué hace	Ejemplo
display	Activa Grid	grid
grid-template-columns	Define las columnas y su tamaño	1fr 1fr 1fr
grid-template-rows	Define las filas y su tamaño	100px auto
grid-template-areas	Da nombres a zonas de la cuadrícula	"header header"
gap	Espacio entre filas y columnas	10px
row-gap / column-gap	Espacio solo entre filas / solo entre columnas	20px
justify-items	Alinea el contenido de las celdas en horizontal	start, center, end, stretch
align-items	Alinea el contenido de las celdas en vertical	start, center, end, stretch
place-items	Atajo de align-items + justify-items	center

Propiedad	Qué hace	Ejemplo
justify-content	Alinea toda la cuadrícula en horizontal	center, space-between
align-content	Alinea toda la cuadrícula en vertical	center, space-between

Propiedades de los elementos (hijos)

Propiedad	Qué hace	Ejemplo
grid-column	En qué columnas se ubica o cuántas ocupa	1 / 3 o span 2
grid-row	En qué filas se ubica o cuántas ocupa	1 / 3 o span 2
grid-area	Le asigna un nombre de área	grid-area: header;
justify-self	Alinea solo ese elemento en horizontal	center
align-self	Alinea solo ese elemento en vertical	center

Explicación de cada propiedad

grid-template-columns: crear columnas

Cada valor es una columna:

Código	Resultado
200px 200px	2 columnas de 200px
1fr 1fr 1fr	3 columnas iguales
200px 1fr	La primera de 200px y la segunda ocupa el resto
repeat(4, 1fr)	4 columnas iguales (forma corta)

repeat(): repetir

En vez de escribir 1fr 1fr 1fr 1fr, se escribe repeat(4, 1fr), que significa "repite 4 veces 1fr".

minmax(): tamaño mínimo y máximo

minmax(200px, 1fr) significa "mínimo 200px y máximo una fracción del espacio".

auto-fit: columnas automáticas

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

En palabras: "pon todas las columnas que quepan, cada una de mínimo 200px". Si la pantalla es grande caben muchas, y si es pequeña caben menos. Así la página se adapta sola al celular **sin necesidad de más código**.

grid-column y grid-row: ocupar varias celdas

Código	Significado
grid-column: 1 / 3;	Va de la línea 1 a la línea 3, es decir, ocupa 2 columnas
grid-column: span 2;	Ocupa 2 columnas desde donde esté
grid-column: 1 / -1;	Ocupa todo el ancho (-1 es la última línea)
grid-row: span 2;	Ocupa 2 filas

grid-template-areas: diseñar con nombres

Se "dibuja" la página con palabras. Cada texto entre comillas es una fila:

CSS

```
grid-template-areas:  
  "header header"  
  "sidebar main"  
  "footer footer";
```

Después, a cada elemento se le dice a qué zona va con grid-area.

Ejemplos:

- **1: Cuadrícula básica de 3 columnas:** grid-template-columns: 1fr 1fr 1fr crea 3 columnas iguales.

```
html

<div class="cuadrícula">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>
```

```
CSS

.cuadrícula {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.cuadrícula div {
  background-color: #ebf4ff;
  padding: 20px;
  text-align: center;
}
```

- **2. Columnas de diferente tamaño con fr**

La columna del medio es el doble de ancha que las otras.

```
html

<div class="anchos">
  <div>1fr</div>
  <div>2fr</div>
  <div>1fr</div>
</div>
```

```
CSS

.anchos {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  gap: 10px;
}
.anchos div {
  background-color: #c6f6d5;
  padding: 20px;
}
```

- **3. Usar repeat():** 4 columnas iguales escritas de forma corta.

html

```
<div class="repetir">
  <div>A</div>
  <div>B</div>
  <div>C</div>
  <div>D</div>
</div>
```

CSS

```
.repetir {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
}
.repetir div {
  background-color: #fefcbf;
  padding: 20px;
}
```

- **4. Galería que se adapta sola (responsive):** en computador se ven varias

html

```
<div class="galeria">
  
  
  
  
</div>
```

CSS

```
.galeria {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
}
.galeria img {
  width: 100%; /* la imagen ocupa todo el ancho de su celda */
}
```

- **5.Un elemento que ocupa 2 columnas:** grid-column: span 2 hace que el primer elemento ocupe 2 columnas.

html

```
<div class="destacado">
  <div class="ancho">Ocupo 2 columnas</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

CSS

```
.destacado {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.destacado div {
  background-color: #fed7d7;
  padding: 20px;
}
.ancho {
  grid-column: span 2;
}
```

- **6.Un elemento que ocupa 2 filas:** grid-row: span 2 hace que el primer elemento ocupe 2 filas.

html

```
<div class="alto">
  <div class="vertical">Ocupo 2 filas</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
```

CSS

```
.alto {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.alto div {
  background-color: #e9d8fd;
  padding: 20px;
}
.vertical {
  grid-row: span 2;
}
```

- **7.Estructura de página con grid-template-areas:** Header arriba, sidebar y main en el medio, footer abajo.

html

```
<div class="pagina">
  <header>Header</header>
  <aside>Sidebar</aside>
  <main>Main</main>
  <footer>Footer</footer>
</div>
```

CSS



```
.pagina {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 10px;
  min-height: 100vh;
}
header { grid-area: header; background-color: #2b6cb0; color: white; padding: 20px; }
aside { grid-area: sidebar; background-color: #38a169; color: white; padding: 20px; }
main { grid-area: main; background-color: #805ad5; color: white; padding: 20px; }
footer { grid-area: footer; background-color: #4a5568; color: white; padding: 20px; }
```

- **8.Centrar con place-items:** con una sola línea se centra en horizontal y vertical.

html

```
<div class="centro">
  <div>Centrado</div>
</div>
```

CSS

```
.centro {
  display: grid;
  place-items: center;
  height: 300px;
  background-color: #f5f5f5;
}
```

- **9. Ubicar elementos por líneas:** se elige exactamente dónde empieza y dónde termina cada element

html

```
<div class="lineas">
  <div class="a">A</div>
  <div class="b">B</div>
  <div class="c">C</div>
</div>
```

CSS

```
.lineas {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: 100px 100px;
  gap: 10px;
}
.lineas div {
  background-color: #bee3f8;
  padding: 10px;
}
.a { grid-column: 1 / 3; grid-row: 1; } /* columnas 1 y 2, fila 1 */
.b { grid-column: 3 / 5; grid-row: 1 / 3; } /* columnas 3 y 4, filas 1 y 2 */
.c { grid-column: 1 / 3; grid-row: 2; } /* columnas 1 y 2, fila 2 */
```

- **Ejemplo 10: Calendario de 7 columnas:** un caso real: los días de la semana.

html

```
<div class="calendario">
  <div class="dia">Lun</div>
  <div class="dia">Mar</div>
  <div class="dia">Mié</div>
  <div class="dia">Jue</div>
  <div class="dia">Vie</div>
  <div class="dia">Sáb</div>
  <div class="dia">Dom</div>
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
</div>
```

CSS

```
.calendario {  
  display: grid;  
  grid-template-columns: repeat(7, 1fr);  
  gap: 5px;  
  text-align: center;  
}  
  
.calendario div {  
  padding: 10px;  
  border: 1px solid #cccccc;  
}  
  
.dia {  
  background-color: #2b6cb0;  
  color: white;  
  font-weight: bold;  
}
```